10 Python Programming Case Studies

Beginner-friendly solutions using the assigned syllabus

Included for every case study

- Problem statement, objectives, and assumptions
- Algorithm and pseudocode
- Colored flowchart using the requested shape/color key
- Four-line Python source code using basic concepts
- Sample test input, console-output screenshot-style panel, and conclusion

Flowchart key	Start / Stop	Input / Output	Process	Decision	Arrows
	Green / light red	Light blue	Light yellow	Light orange	Dark grey

Note: Amounts, thresholds, taxes, and policies are illustrative classroom assumptions. Real-world rules may differ.

1. University Scholarship Allocation

Problem Statement

Evaluate applicants using aggregate marks, minimum subject scores, attendance, and family income. Rank eligible applicants and break ties using aggregate marks, then attendance.

Objectives

• Check eligibility consistently.

• Rank eligible applicants using fixed tie-break rules.

• Display eligibility and a rank score.

Assumptions

• Marks and attendance are percentages from 0 to 100.

• Eligibility: aggregate ≥ 80, every subject ≥ 60, attendance ≥ 75%, income ≤ ■3,00,000.

• This four-line example evaluates one applicant; multiple applicants can be stored as tuples and sorted.

Algorithm

1. Read aggregate, subject marks, attendance, and income.

2. Check all thresholds using a Boolean expression.

3. Print eligible or not eligible.

4. For multiple applicants, sort eligible records by aggregate descending, then attendance descending.

Pseudocode

INPUT aggregate, subject1, subject2, subject3, attendance, income

eligible ← aggregate ≥ 80 AND min(subjects) ≥ 60 AND attendance ≥ 75 AND income ≤ 300000

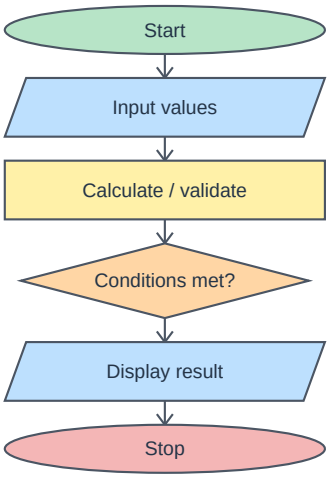
IF eligible THEN OUTPUT “Eligible” ELSE OUTPUT “Not eligible”

For ranking: sort by (aggregate, attendance) descending

```
INPUT aggregate, subject1, subject2, subject3, attendance, income
eligible ← aggregate ≥ 80 AND min(subjects) ≥ 60 AND attendance ≥ 75 AND income ≤ 300000
IF eligible THEN OUTPUT “Eligible” ELSE OUTPUT “Not eligible”
For ranking: sort by (aggregate, attendance) descending
```

1. University Scholarship Allocation — Flowchart, Code & Test

Flowchart



General flow; condition outcomes are described in the algorithm.

Python Source Code (exactly four lines)

```
aggregate=86; subjects=(78,64,91); attendance=88; income=240000
eligible=aggregate>=80 and min(subjects)>=60 and attendance>=75 and income<=300000
print('Eligible' if eligible else 'Not eligible')
print('Rank score:', aggregate, 'Attendance:', attendance)
```

Test Case

aggregate=86, subjects=(78,64,91), attendance=88, income=240000

Output Screenshot

```
Terminal / Console Output

Eligible
Rank score: 86 Attendance: 88
```

Conclusion

The program applies all four eligibility conditions. For a full university list, store applicant tuples in a list and sort eligible applicants by aggregate and attendance.

2. Progressive Electricity Billing

Problem Statement

Calculate an electricity bill using consumption slabs, fixed charges, surcharge, and rebate. Compare bills across months and flag a large increase.

Objectives

• Calculate slab-wise energy charges.

• Add fixed charge and surcharge, then apply a rebate.

• Identify a month-to-month increase above a chosen limit.

Assumptions

• First 100 units at ■2/unit, next 100 at ■3/unit, remaining at ■5/unit.

• Fixed charge ■50; surcharge 5%; rebate ■20 if usage is below 100 units.

• An increase greater than 30% compared with last month is unusual.

Algorithm

1. Read current and previous bills/units.

2. Calculate each progressive slab charge.

3. Add fixed charge and surcharge; apply eligible rebate.

4. Compare current bill with previous bill and print an alert if the increase exceeds 30%.

Pseudocode

INPUT units, previous_bill

energy ← slab_charge(units)

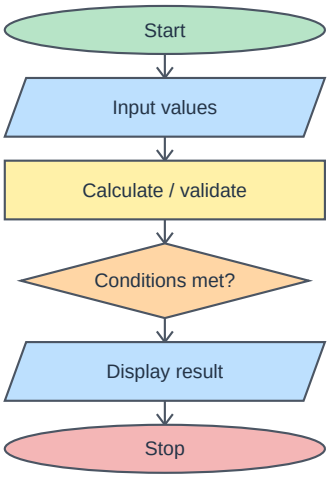
bill ← (energy + 50) × 1.05; IF units < 100 THEN bill ← bill – 20

IF bill > previous_bill × 1.30 THEN OUTPUT “Unusual increase”

```
INPUT units, previous_bill
energy ← slab_charge(units)
bill ← (energy + 50) × 1.05; IF units < 100 THEN bill ← bill – 20
IF bill > previous_bill × 1.30 THEN OUTPUT “Unusual increase”
```

2. Progressive Electricity Billing — Flowchart, Code & Test

Flowchart



General flow; condition outcomes are described in the algorithm.

Python Source Code (exactly four lines)

```
u=180; prev_bill=350
energy=min(u,100)*2+min(max(u-100,0),100)*3+max(u-200,0)*5
bill=(energy+50)*1.05-(20 if u<100 else 0)
print('Bill: ■%.2f'%bill, '| Unusual increase:', bill>prev_bill*1.30)
```

Test Case

Current usage=180 units; previous month bill=■350

Output Screenshot

```
Terminal / Console Output

Energy charge = ■400.00
Bill = ■472.50
Unusual increase: True
```

Conclusion

The slab calculation charges different rates for each unit range and checks whether the current bill is unusually higher than the previous bill.

3. Hospital Billing and Patient Classification

Problem Statement

Calculate consultation, diagnostic, and treatment charges, apply a concession, validate the bill, and classify the patient under a sample payment policy.

Objectives

• Add service charges accurately.

• Apply a concession only when its condition is met.

• Validate charges and classify the payment category.

Assumptions

• All charges must be non-negative.

• A 10% concession applies when age is 60 or above.

• Bill under ■2,000 = Basic; otherwise Standard. This is an example policy.

Algorithm

1. Read age and service charges.

2. Validate that each charge is non-negative.

3. Add charges and apply senior concession if eligible.

4. Print total and payment category.

Pseudocode

INPUT age, consultation, diagnostics, treatment

IF any charge < 0 THEN OUTPUT “Invalid charges”

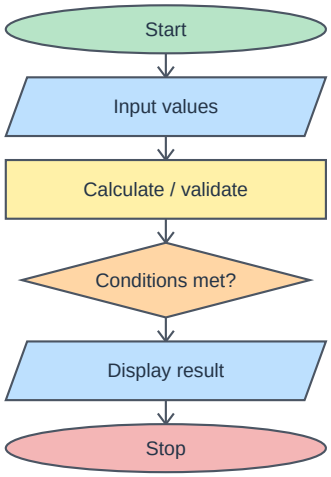
total ← sum(charges); IF age ≥ 60 THEN total ← total × 0.90

category ← Basic IF total < 2000 ELSE Standard; OUTPUT total, category

```
INPUT age, consultation, diagnostics, treatment
IF any charge < 0 THEN OUTPUT “Invalid charges”
total ← sum(charges); IF age ≥ 60 THEN total ← total × 0.90
category ← Basic IF total < 2000 ELSE Standard; OUTPUT total, category
```

3. Hospital Billing and Patient Classification — Flowchart, Code & Test

Flowchart



General flow; condition outcomes are described in the algorithm.

Python Source Code (exactly four lines)

```
age=65; consultation=500; diagnostics=900; treatment=1200
valid=min(consultation,diagnostics,treatment)>=0
total=(consultation+diagnostics+treatment)*(0.9 if age>=60 else 1)
print('Invalid charges' if not valid else f'Total: ■{total:.2f} | Category: {"Basic" if total<2000 else "Standard"}')
```

Test Case

Age=65; consultation=■500; diagnostics=■900; treatment=■1,200

Output Screenshot

```
Terminal / Console Output

Subtotal = ■2,600.00
Senior concession = 10%
Final total = ■2,340.00
Category = Standard
```

Conclusion

The program validates the charges, applies the concession conditionally, and classifies the bill using the stated threshold.

4. Employee Payroll and Deduction Analysis

Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives, then apply progressive deductions and display a salary statement.

Objectives

• Calculate gross earnings.

• Apply a simple progressive deduction policy.

• Show gross pay, deduction, and take-home pay.

Assumptions

• Gross = basic + allowances + overtime + incentives.

• Deduction: 5% on first ■20,000 and 10% on the amount above ■20,000.

• This is an educational example; actual payroll rules vary.

Algorithm

1. Read salary components.

2. Add components to get gross pay.

3. Calculate deduction using two bands.

4. Subtract deduction from gross and display a formatted statement.

Pseudocode

INPUT basic, allowances, overtime, incentives

gross ← basic + allowances + overtime + incentives

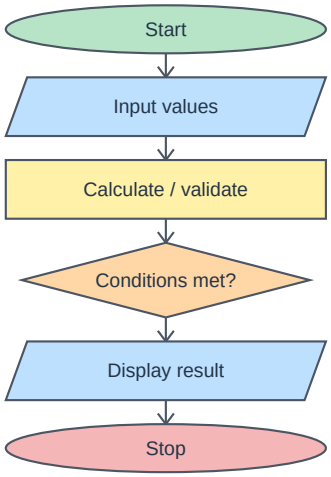
deduction ← 0.05 × min(gross, 20000) + 0.10 × max(gross – 20000, 0)

OUTPUT gross, deduction, gross – deduction

```
INPUT basic, allowances, overtime, incentives
gross ← basic + allowances + overtime + incentives
deduction ← 0.05 × min(gross, 20000) + 0.10 × max(gross – 20000, 0)
OUTPUT gross, deduction, gross – deduction
```

4. Employee Payroll and Deduction Analysis — Flowchart, Code & Test

Flowchart



General flow; condition outcomes are described in the algorithm.

Python Source Code (exactly four lines)

```
basic=25000; allowances=5000; overtime=2000; incentives=1500
gross=basic+allowances+overtime+incentives
deduction=0.05*min(gross,20000)+0.10*max(gross-20000,0)
print(f'Gross: ■{gross:.2f} | Deduction: ■{deduction:.2f} | Net: ■{gross-deduction:.2f}')
```

Test Case

Basic=■25,000; allowances=■5,000; overtime=■2,000; incentives=■1,500

Output Screenshot

```
Terminal / Console Output

Gross salary = ■33,500.00
Deduction = ■2,350.00
Take-home salary = ■31,150.00
```

Conclusion

The salary statement separates gross earnings, progressive deductions, and net salary.

5. Multi-Plan Telecommunications Cost Optimization

Problem Statement

Compare mobile plans against usage, validity, and budget constraints, calculate excess data charges, and identify the cheapest eligible plan.

Objectives

• Calculate total estimated plan costs.

• Check validity and budget constraints.

• Select the lowest-cost eligible plan.

Assumptions

• Plan A: ■199, 2 GB, 28 days; Plan B: ■299, 5 GB, 56 days.

• Extra data costs ■20/GB; required validity is 28 days and budget is ■350.

• Two plan tuples are used for this compact example.

Algorithm

1. Store plans as tuples (name, price, data, validity).

2. Calculate excess-data cost and total cost for each plan.

3. Keep plans meeting validity and budget.

4. Print the cheapest eligible plan.

Pseudocode

plans ← [(A,199,2,28), (B,299,5,56)]

FOR each plan: cost ← price + max(usage – data, 0) × 20

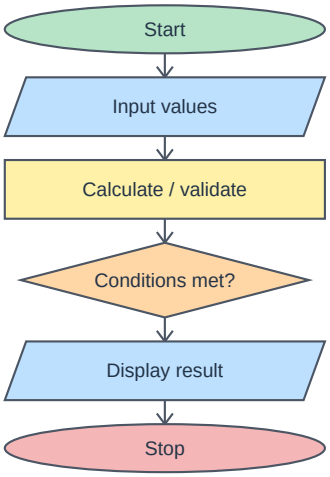
eligible ← validity ≥ required_days AND cost ≤ budget

OUTPUT eligible plan with minimum cost

```
plans ← [(A,199,2,28), (B,299,5,56)]
FOR each plan: cost ← price + max(usage – data, 0) × 20
eligible ← validity ≥ required_days AND cost ≤ budget
OUTPUT eligible plan with minimum cost
```

5. Multi-Plan Telecommunications Cost Optimization — Flowchart, Code & Test

Flowchart



General flow; condition outcomes are described in the algorithm.

Python Source Code (exactly four lines)

```
usage=3; days=28; budget=350; plans=[('A',199,2,28),('B',299,5,56)]
costs=[(n,p+max(usage-d,0)*20,v) for n,p,d,v in plans if v>=days and p+max(usage-d,0)*20<=budget]
best=min(costs,key=lambda x:x[1]) if costs else None
print('Best plan:', best if best else 'No eligible plan')
```

Test Case

Usage=3 GB; required validity=28 days; budget=■350

Output Screenshot

```
Terminal / Console Output

Plan A total = ■219 (■20 excess data)
Plan B total = ■299
Most economical eligible plan = A, ■219
```

Conclusion

The program compares estimated total costs rather than base prices alone and selects the cheapest plan satisfying the constraints.

6. Railway Fare and Reservation Calculator

Problem Statement

Calculate passenger fares using distance, travel class, age concessions, reservation fees, and cancellation penalties.

Objectives

• Calculate base fare from distance and class.

• Apply a sample age concession and reservation fee.

• Estimate refund after cancellation penalty.

Assumptions

• Sleeper rate ■1.20/km; AC rate ■2.00/km.

• Age 60 or above receives a 20% sample concession.

• Reservation fee ■40; cancellation penalty is 25% of discounted fare.

Algorithm

1. Read distance, class, age, and cancellation choice.

2. Choose class rate and calculate base fare.

3. Apply age concession, then add reservation fee.

4. If cancelled, subtract the penalty and display the refund.

Pseudocode

INPUT distance, class, age, cancelled

rate ← 1.2 IF Sleeper ELSE 2.0; fare ← distance × rate

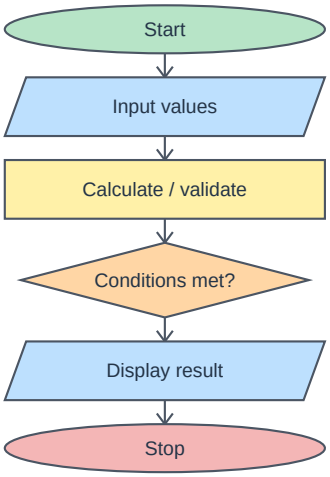
IF age ≥ 60 THEN fare ← fare × 0.80; total ← fare + 40

IF cancelled THEN refund ← total – 0.25 × fare; OUTPUT total, refund

```
INPUT distance, class, age, cancelled
rate ← 1.2 IF Sleeper ELSE 2.0; fare ← distance × rate
IF age ≥ 60 THEN fare ← fare × 0.80; total ← fare + 40
IF cancelled THEN refund ← total – 0.25 × fare; OUTPUT total, refund
```

6. Railway Fare and Reservation Calculator — Flowchart, Code & Test

Flowchart



General flow; condition outcomes are described in the algorithm.

Python Source Code (exactly four lines)

```
distance=350; travel_class='AC'; age=65; cancelled=True
rate=1.2 if travel_class=='Sleeper' else 2.0; fare=distance*rate
fare=fare*(0.8 if age>=60 else 1); total=fare+40; refund=total-0.25*fare if cancelled else 0
print(f'Fare: ■{total:.2f} | Refund on cancellation: ■{refund:.2f}')
```

Test Case

Distance=350 km; AC class; age=65; cancellation requested

Output Screenshot

```
Terminal / Console Output

Base fare = ■700.00
Age concession = ■140.00
Total paid = ■600.00
Estimated cancellation refund = ■460.00
```

Conclusion

The calculator demonstrates class rates, a conditional concession, reservation fee, and a stated cancellation rule. Real railway rules may differ.

7. Retail Billing with Multiple Discount Rules

Problem Statement

Prepare a retail bill using product quantities, category tax, threshold discounts, and a maximum concession limit.

Objectives

• Calculate line totals from quantity and unit price.

• Apply a threshold discount with a maximum limit.

• Add tax and print an itemized invoice total.

Assumptions

• Electronics tax is 18%; groceries tax is 5%.

• A 10% discount applies when subtotal is at least ■2,000, capped at ■300.

• Discount is applied before tax.

Algorithm

1. Store product names, quantities, and prices in a list.

2. Calculate subtotal and choose the discount.

3. Limit discount to ■300 and calculate taxable amount.

4. Apply category tax and print final total.

Pseudocode

subtotal ← sum(quantity × price)

discount ← min(10% × subtotal, 300) IF subtotal ≥ 2000 ELSE 0

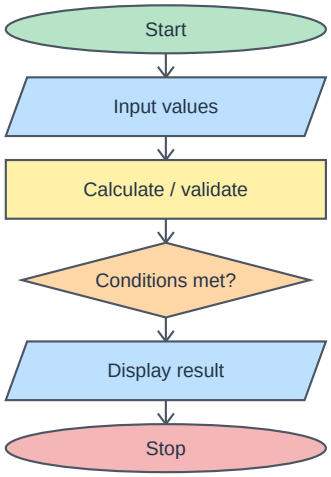
tax_rate ← 18% IF Electronics ELSE 5%

total ← (subtotal – discount) × (1 + tax_rate); OUTPUT total

```
subtotal ← sum(quantity × price)
discount ← min(10% × subtotal, 300) IF subtotal ≥ 2000 ELSE 0
tax_rate ← 18% IF Electronics ELSE 5%
total ← (subtotal – discount) × (1 + tax_rate); OUTPUT total
```

7. Retail Billing with Multiple Discount Rules — Flowchart, Code & Test

Flowchart



General flow; condition outcomes are described in the algorithm.

Python Source Code (exactly four lines)

```
items=[('Headphones',2,900),('Cable',1,300)]; category='Electronics'
subtotal=sum(q*p for name,q,p in items); discount=min(subtotal*0.10,300) if subtotal>=2000 else 0
rate=0.18 if category=='Electronics' else 0.05; total=(subtotal-discount)*(1+rate)
print('Subtotal: ■%.2f | Discount: ■%.2f | Tax: ■%.2f | Total:
■%.2f'%(subtotal,discount,(subtotal-discount)*rate,total))
```

Test Case

2 headphones at ■900 each and 1 cable at ■300; Electronics category

Output Screenshot

```
Terminal / Console Output

Subtotal = ■2,100.00
Discount = ■210.00
Tax = ■340.20
Invoice total = ■2,230.20
```

Conclusion

The invoice applies a threshold discount before tax and limits the concession to the defined maximum.

8. Manufacturing Cost and Profitability Analysis

Problem Statement

Calculate material, labour, wastage, and machine costs, determine unit cost and selling price, and check a profit target.

Objectives

• Add production costs including wastage.

• Calculate total revenue and profit.

• Check whether the profit target is achieved.

Assumptions

• Material cost ■500 per unit for 100 units.

• Wastage adds 8% to material cost; labour ■12,000 and machine cost ■5,000.

• Selling price ■750 per unit; target profit ■15,000.

Algorithm

1. Calculate material cost and wastage.

2. Add labour and machine costs.

3. Calculate revenue and profit.

4. Compare profit with target and display the result.

Pseudocode

material ← quantity × material_rate; wastage ← material × wastage_rate

total_cost ← material + wastage + labour + machine

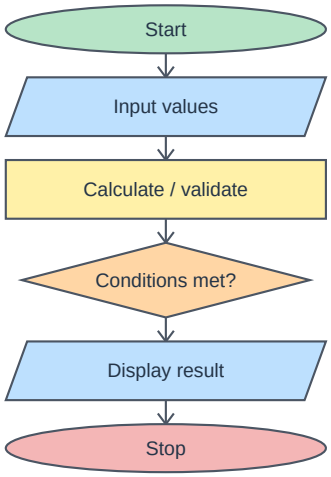
revenue ← quantity × selling_price; profit ← revenue – total_cost

OUTPUT profit and whether profit ≥ target

```
material ← quantity × material_rate; wastage ← material × wastage_rate
total_cost ← material + wastage + labour + machine
revenue ← quantity × selling_price; profit ← revenue – total_cost
OUTPUT profit and whether profit ≥ target
```

8. Manufacturing Cost and Profitability Analysis — Flowchart, Code & Test

Flowchart



General flow; condition outcomes are described in the algorithm.

Python Source Code (exactly four lines)

```
qty=100; material_rate=500; labour=12000; machine=5000; selling_price=750
material=qty*material_rate; wastage=material*0.08; cost=material+wastage+labour+machine
revenue=qty*selling_price; profit=revenue-cost; target=15000
print(f'Cost: ■{cost:.2f} | Revenue: ■{revenue:.2f} | Profit: ■{profit:.2f} | Target met: {profit>=target}')
```

Test Case

100 units; material ■500/unit; labour ■12,000; machine ■5,000; selling price ■750/unit

Output Screenshot

```
Terminal / Console Output

Material = ■50,000.00
Wastage = ■4,000.00
Total cost = ■71,000.00
Revenue = ■75,000.00; Profit = ■4,000.00
Target met: False
```

Conclusion

The calculation includes material wastage and operating costs, then checks actual profit against the required target.

9. Multi-Currency Travel Budget Analysis

Problem Statement

Convert travel expenses using exchange rates, add conversion fees, compare spending with a budget, and identify the largest expense category.

Objectives

• Convert foreign expenses to local currency.

• Add a conversion fee.

• Find the largest category and check the budget.

Assumptions

• Expenses are provided in one foreign currency.

• Example exchange rate is ■83 per foreign-currency unit, with a 2% conversion fee.

• Budget is ■50,000; largest category is based on converted amount.

Algorithm

1. Store category and amount in a list of tuples.

2. Convert each amount using the exchange rate.

3. Add 2% fee and calculate total.

4. Find largest category and compare total with budget.

Pseudocode

expenses ← [(Hotel,300), (Food,100), (Transport,80)]

FOR each expense: converted ← amount × exchange_rate

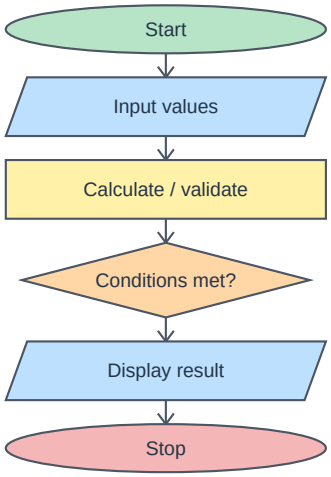
total ← sum(converted) × 1.02; largest ← category with maximum amount

OUTPUT total, largest, and budget status

```
expenses ← [(Hotel,300), (Food,100), (Transport,80)]
FOR each expense: converted ← amount × exchange_rate
total ← sum(converted) × 1.02; largest ← category with maximum amount
OUTPUT total, largest, and budget status
```

9. Multi-Currency Travel Budget Analysis — Flowchart, Code & Test

Flowchart



General flow; condition outcomes are described in the algorithm.

Python Source Code (exactly four lines)

```
expenses=[('Hotel',300),('Food',100),('Transport',80)]; rate=83; budget=50000
converted=[(name,amount*rate) for name,amount in expenses]
total=sum(amount for name,amount in converted)*1.02; largest=max(converted,key=lambda x:x[1])
print(f'Total with fee: ■{total:.2f} | Largest: {largest[0]} | Within budget: {total<=budget}')
```

Test Case

Hotel=300, Food=100, Transport=80 foreign currency units; rate ■83; budget ■50,000

Output Screenshot

```
Terminal / Console Output

Converted subtotal = ■39,840.00
2% fee = ■796.80
Total = ■40,636.80
Largest category = Hotel
Within budget: True
```

Conclusion

The program converts each category, adds a fee, identifies the largest expense, and checks the budget.

10. Campus Event Financial Planning

Problem Statement

Calculate event costs, participant contributions, sponsorship shortfall, and the minimum registrations required to break even.

Objectives

• Calculate fixed and per-participant costs.

• Determine funding shortfall after sponsorship and registrations.

• Estimate minimum registrations to break even.

Assumptions

• Venue ■20,000; equipment ■8,000; transport ■5,000; certificates ■50/person; catering ■250/person.

• Sponsorship ■15,000 and registration fee ■400/person.

• Per-person contribution after variable cost is ■100.

Algorithm

1. Calculate fixed costs and per-participant cost.

2. Calculate total cost and funding after sponsorship and registration income.

3. Calculate shortfall if funding is less than cost.

4. Calculate minimum registrations using fixed costs minus sponsorship divided by contribution per person.

Pseudocode

fixed ← venue + equipment + transport; per_person ← catering + certificate

total_cost ← fixed + participants × per_person

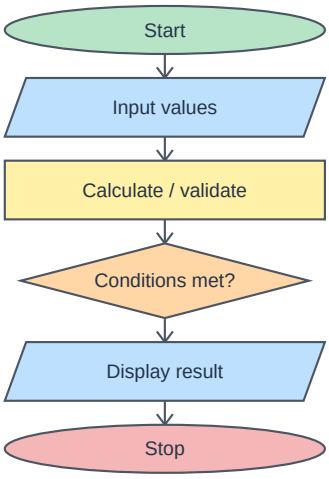
shortfall ← max(total_cost – sponsorship – participants × fee, 0)

minimum ← ceil((fixed – sponsorship)/(fee – per_person)); OUTPUT results

```
fixed ← venue + equipment + transport; per_person ← catering + certificate
total_cost ← fixed + participants × per_person
shortfall ← max(total_cost – sponsorship – participants × fee, 0)
minimum ← ceil((fixed – sponsorship)/(fee – per_person)); OUTPUT results
```

10. Campus Event Financial Planning — Flowchart, Code & Test

Flowchart



General flow; condition outcomes are described in the algorithm.

Python Source Code (exactly four lines)

```
venue=20000; equipment=8000; transport=5000; people=120; catering=250; certificate=50
fixed=venue+equipment+transport; per_person=catering+certificate; total=fixed+people*per_person
sponsor=15000; fee=400; shortfall=total-sponsor-people*fee; minimum=(fixed-sponsor)//(fee-per_person)+1
print(f'Total cost: ■{total} | Funding shortfall: ■{max(shortfall,0)} | Break-even registrations: {minimum}')
```

Test Case

120 participants; sponsorship ■15,000; registration fee ■400 per participant

Output Screenshot

```
Terminal / Console Output

Fixed cost = ■33,000.00
Variable cost = ■36,000.00
Total cost = ■69,000.00
Registration income = ■48,000.00; sponsorship = ■15,000.00
Funding shortfall = ■6,000.00
Break-even registrations = 180
```

Conclusion

The event budget separates fixed and variable costs and estimates the registrations required to cover costs after sponsorship.